BCSE302P - Database Systems Lab

Review 1: Problem Definition, Requirement Analysis & Database Design

Project: *Student Attendance Management System Using MongoDB*

Slot: D2

Team Members: Ayush Naugariya (24BDS0076) , Abhishek VS (24BCE0404)

1. Problem Statement

In educational institutions, student attendance is often managed using manual registers or basic digital systems. These approaches can make attendance recording, updating, monitoring, and generating reports time-consuming and error prone. Faculty members may also find it difficult to quickly identify students with low attendance, analyze attendance subject-wise, and maintain historical attendance records efficiently.

The proposed **Student Attendance Management System** is a web-based application that provides a centralized platform for managing student, faculty, subject, class, and attendance information. The system will use **MongoDB, a NoSQL document-oriented database**, to store and manage attendance-related data.

The system will allow authorized users, particularly administrators and faculty members, to perform operations such as adding and managing students, subjects, and classes, recording attendance, updating attendance records, and generating attendance analytics.

2. Objectives

• To develop a centralized system for managing student attendance.

• To store student and attendance information efficiently using MongoDB.

• To provide CRUD operations for managing students, subjects, classes, and attendance records.

• To enable faculty members to record and update attendance digitally.

• To calculate student attendance percentages automatically.

• To generate subject-wise and student-wise attendance reports.

• To identify students whose attendance falls below a specified threshold.

• To demonstrate NoSQL concepts such as document-based storage, aggregation, indexing, and schema design.

• To provide a responsive and user-friendly interface.

• To implement basic authentication so that only authorized users can access management functions.

3. Scope of the Project

The system will cover:

• Student registration and management

• Faculty management

• Subject management

• Class/session management

• Attendance recording and modification

• Attendance history

• Student-wise and subject-wise attendance reports

• Attendance percentage calculation

• Identification of students with low attendance

• Dashboard and basic analytics

• User authentication

• MongoDB database operations

• Search and filtering of records

4. Target Users

Administrator

Manage students, faculty, subjects, classes, user accounts, attendance records, and reports.

Faculty

Login, view assigned subjects/classes, view student lists, mark attendance, modify attendance when required, and view attendance statistics.

Student

View personal attendance, subject-wise attendance, attendance history, and attendance status.

5. Functional Requirements

Authentication

• User login

• Role-based access

• Secure password storage

• Logout

Student Management

• Add, view, update, and delete students

• Search students

• Filter by department/year/section

Faculty Management

• Add, view, update, and delete faculty

Subject Management

• Add, update, delete, and view subjects

• Assign subjects to faculty

Class Management

• Create attendance sessions

• Select subject, date, time, and classroom

• View previous sessions

Attendance Management

• Mark Present/Absent

• Update attendance

• Delete attendance records

• View attendance by date and subject

Reports & Analytics

• Overall attendance percentage

• Subject-wise attendance

• Monthly attendance

• Daily attendance statistics

• Students below threshold

6. Non-Functional Requirements

Performance: Efficient retrieval of attendance records and reports using suitable MongoDB indexes.

Usability: Simple and easy-to-use interface for faculty and administrators.

Security: Only authenticated users can access protected operations.

Scalability: Database design should support increasing numbers of students, subjects, classes, and attendance records.

Availability: Application should be accessible whenever the application/server is available.

Responsiveness: User interface should work properly on desktop, tablet, and mobile screen sizes.

7. Why MongoDB?

MongoDB is selected as the NoSQL database because it is a document-oriented database that stores data in flexible JSON-like documents. This is suitable for an attendance system because student, subject, class, and attendance information can be represented naturally as documents.

Flexible Document Structure: Student and attendance information can be stored as documents without a rigid relational table structure.

Scalability: The system can accommodate increasing numbers of students and attendance records.

Aggregation Framework: Aggregation pipelines can be used for attendance percentages, subject-wise reports, monthly reports, daily statistics, and low-attendance identification.

Indexing: Indexes can improve frequently used queries such as registration number, student ID, subject, and date.

Web Application Compatibility: MongoDB integrates naturally with a Node.js/Express backend and JSON-based application data.

NoSQL Concepts: The project demonstrates document storage, aggregation, indexing, and NoSQL schema design.

8. Database Design

The proposed database contains six MongoDB collections: **Users, Students, Faculty, Subjects, Classes, and Attendance**.

Collection	Purpose	Important Fields
Users	Authentication and roles	_id, name, email, password, role, createdAt
Students	Student master data	_id, regNo, name, email, department, year, section, semester
Faculty	Faculty information	_id, facultyId, name, email, department
Subjects	Subjects and faculty assignment	_id, subjectCode, subjectName, facultyId, semester, credits
Classes	Individual class/attendance sessions	_id, subjectId, facultyId, date, startTime, endTime, room
Attendance	Core attendance records	_id, classId, studentId, subjectId, date, status, markedBy

9. Sample Document Designs

Students

```
{
  "_id": ObjectId("..."),
  "regNo": "24BDS0076",
  "name": "Ayush Naugariya",
  "department": "Computer Science",
  "year": 3,
  "section": "D2",
  "semester": 5
}
```

Subjects

```
{
  "_id": ObjectId("..."),
  "subjectCode": "BCSE302P",
  "subjectName": "Database Systems Lab",
  "facultyId": ObjectId("..."),
  "semester": 5,
  "credits": 2
}
```

Classes

```
{
  "_id": ObjectId("..."),
  "subjectId": ObjectId("..."),
  "facultyId": ObjectId("..."),
  "date": ISODate("2026-08-21"),
  "startTime": "10:00",
  "endTime": "12:00",
  "room": "SJT-302"
}
```

Attendance

```
{
  "_id": ObjectId("..."),
  "classId": ObjectId("..."),
  "studentId": ObjectId("..."),
  "subjectId": ObjectId("..."),
  "date": ISODate("2026-08-21"),
  "status": "Present",
  "markedBy": ObjectId("...")
}
```

10. Collection References / Relationships

• Subjects.facultyId → Faculty._id

• Classes.subjectId → Subjects._id

• Classes.facultyId → Faculty._id

• Attendance.classId → Classes._id

• Attendance.studentId → Students._id

• Attendance.subjectId → Subjects._id

• Attendance.markedBy → Users._id

11. ER-to-NoSQL Mapping

Traditional Entity	MongoDB Collection
Student	students
Faculty	faculty
Subject	subjects
Class	classes
Attendance	attendance
User	users

The relational-style entities are represented as MongoDB collections, while associations between documents are maintained using ObjectId references. This design keeps the data model understandable while allowing MongoDB's document-oriented and aggregation capabilities to be demonstrated.

12. Sample Dataset Design

Collection	Planned Records
Users	10
Students	60
Faculty	10
Subjects	10
Classes	30
Attendance	500+
Total	620+

13. Planned CRUD Operations

• Create, read, update, and delete student records.

• Create, read, update, and delete faculty records.

• Create, read, update, and delete subject records.

• Create and manage class/attendance sessions.

• Create, read, update, and delete attendance records.

14. Planned Aggregation Queries

• Student attendance percentage.

• Subject-wise attendance percentage.

• Monthly attendance report.

• Students below the attendance threshold.

• Daily present/absent statistics.

• Additional analytics such as section-wise attendance and attendance trends.

15. Indexing Plan

• Unique index on students.regNo for fast student lookup and uniqueness.

• Compound index on attendance.studentId, attendance.subjectId, and attendance.date for attendance - history queries.

Additional date index may be used for date-based attendance reports.

16. Proposed Technology Stack

Layer	Technology
Frontend	React.js + Tailwind CSS
Backend	Node.js + Express.js
Database	MongoDB / MongoDB Atlas
ODM	Mongoose
Authentication	JWT
Charts	Recharts